Day 3 — Data Structures and Algorithms (DSA)

Topics Covered

1. Functions in C++
2. Function Parameters
3. Parameter Passing Methods
4. Pass by Value
5. Pass by Address (Pointers)
6. Pass by Reference
7. Arrays as Function Parameters
8. Binary Search Revision
9. LeetCode Problem #704 — Binary Search
10. LeetCode Problem #35 — Search Insert Position
11. Optional: LeetCode Problem #278 — First Bad Version

---

1. Functions in C++

Definition

A function is a reusable block of code designed to perform a specific task. Functions help make programs modular, readable, reusable, and easier to debug.

Syntax

```
return_type function_name(parameters) {
    // Function body
    return value;
}
```

Example

```
#include <iostream>
using namespace std;

int add(int a, int b) {
    return a + b;
```

```
}

int main() {
    int result = add(10, 20);
    cout << result;
    return 0;
}
```

Output:

```
30
```

Explanation

- `int`: Return type of the function.
- `add`: Function name.
- `int a, int b`: Parameters.
- `return a + b`: Returns the sum.
- `add(10, 20)`: Calls the function with arguments 10 and 20.

Advantages of Functions

- Code reusability.
- Better readability.
- Easier debugging and testing.
- Reduced code duplication.
- Improved modularity.

---

2. Parameters and Arguments

A parameter is a variable declared in a function definition. An argument is the actual value passed to the function when it is called.

Example:

```
int square(int n) {
    return n * n;
}

int main() {
    cout << square(5);
    return 0;
```

}

Here:

- `n` is a parameter.
- `5` is an argument.
- Output is `25`.

---

3. Parameter Passing Methods

In C++, common parameter-passing methods include:

1. Pass by Value
2. Pass by Address
3. Pass by Reference

A. Pass by Value

A copy of the original variable is passed to the function. Changes to the parameter do not affect the original variable.

```
#include <iostream>
using namespace std;

void change(int x) {
    x = 50;
}

int main() {
    int a = 10;
    change(a);
    cout << a;
    return 0;
}
```

Output:

10

Explanation:
The function modifies its local copy `x`. The original variable `a` remains 10.

B. Pass by Address

The memory address of a variable is passed using a pointer. The function can modify the original variable through that pointer.

```
#include <iostream>
using namespace std;

void change(int *x) {
    *x = 50;
}

int main() {
    int a = 10;
    change(&a);
    cout << a;
    return 0;
}
```

Output:

50

Explanation:

- `&a` obtains the address of `a`.
- `int *x` declares a pointer parameter.
- `*x = 50` modifies the value stored at that address.

C. Pass by Reference

A reference parameter acts as another name for the original variable.

```
#include <iostream>
using namespace std;

void change(int &x) {
    x = 50;
}

int main() {
    int a = 10;
    change(a);
    cout << a;
    return 0;
```

```
}
```

Output:

50

Explanation:
`int &x` creates a reference parameter. Changing `x` changes the original variable `a`.

---

4. Difference Between Pass by Value, Address, and Reference

Feature	Pass by Value	Pass by Address	Pass by Reference
Parameter	Normal variable	Pointer	Reference
Argument	Value or variable	Address using `&`	Original variable
Changes original variable?	No	Yes, if used to modify it	Yes
Syntax example	`int x`	`int *x`	`int &x`
Access original value	Local copy	Dereference with `*x`	Directly through `x`

Important: Pass by address still passes a pointer value by value. The pointer lets the function access the original object.

---

5. Swap Two Numbers

Using Pass by Reference

```
#include <iostream>
using namespace std;

void swapNumbers(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}
```

```
int main() {
    int x = 10, y = 20;

    swapNumbers(x, y);

    cout << x << " " << y;
    return 0;
}
```

Output:

```
20 10
```

Using Pointers

```
#include <iostream>
using namespace std;

void swapNumbers(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 10, y = 20;

    swapNumbers(&x, &y);

    cout << x << " " << y;
    return 0;
}
```

Output:

```
20 10
```

---

6. Arrays as Function Parameters

Arrays can be passed to functions so that the function can process their elements.

Example:

```
#include <iostream>
using namespace std;

void displayArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
}

int main() {
    int numbers[] = {10, 20, 30, 40, 50};
    displayArray(numbers, 5);
    return 0;
}
```

Output:

10 20 30 40 50

Important points:

- The array name usually converts to a pointer to its first element when passed to a function.
- The function does not automatically receive the array's length.
- Pass the array size separately.
- Changes to elements through the parameter can affect the original array.

---

7. Binary Search Revision

Definition

Binary Search is an efficient searching algorithm that repeatedly divides a sorted search range into two halves.

Requirement

The input must be sorted in ascending or descending order, and the comparison logic must match that ordering.

Algorithm

1. Initialize `low` and `high`.

2. Calculate `mid`.
3. Compare the middle element with the target.
4. If equal, return the index.
5. If the target is greater, search the right half.
6. If the target is smaller, search the left half.
7. Continue until the target is found or the range becomes empty.

C++ Implementation

```
#include <iostream>
using namespace std;

int binarySearch(int arr[], int n, int target) {
    int low = 0;
    int high = n - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    return -1;
}

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = 5;

    cout << binarySearch(arr, n, 40);
    return 0;
}
```

Output:

```
3
```

The returned index is 3 because C++ array indexing starts at 0.

Complexity

- Time: O(log n)
- Auxiliary space: O(1) for the iterative implementation.

---

8. LeetCode #704 — Binary Search

Problem

Given a sorted array of distinct integers and a target, return the index of the target. If the target is not present, return -1.

C++ Solution

```
class Solution {
public:
    int search(vector<int>& nums, int target) {
        int low = 0;
        int high = nums.size() - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (nums[mid] == target) {
                return mid;
            } else if (nums[mid] < target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }

        return -1;
    }
};
```

Key Learning

- Binary Search works on sorted arrays.
- `low`, `high`, and `mid` control the search range.
- Each iteration eliminates approximately half of the remaining elements.

---

9. LeetCode #35 — Search Insert Position

Problem

Given a sorted array of distinct integers and a target, return the index if the target exists. Otherwise, return the index where it should be inserted to maintain sorted order.

C++ Solution

```
class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {
        int low = 0;
        int high = nums.size() - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (nums[mid] == target) {
                return mid;
            } else if (nums[mid] < target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }

        return low;
    }
};
```

Example

Input:

nums = [1, 3, 5, 6]
target = 2

Output:

1

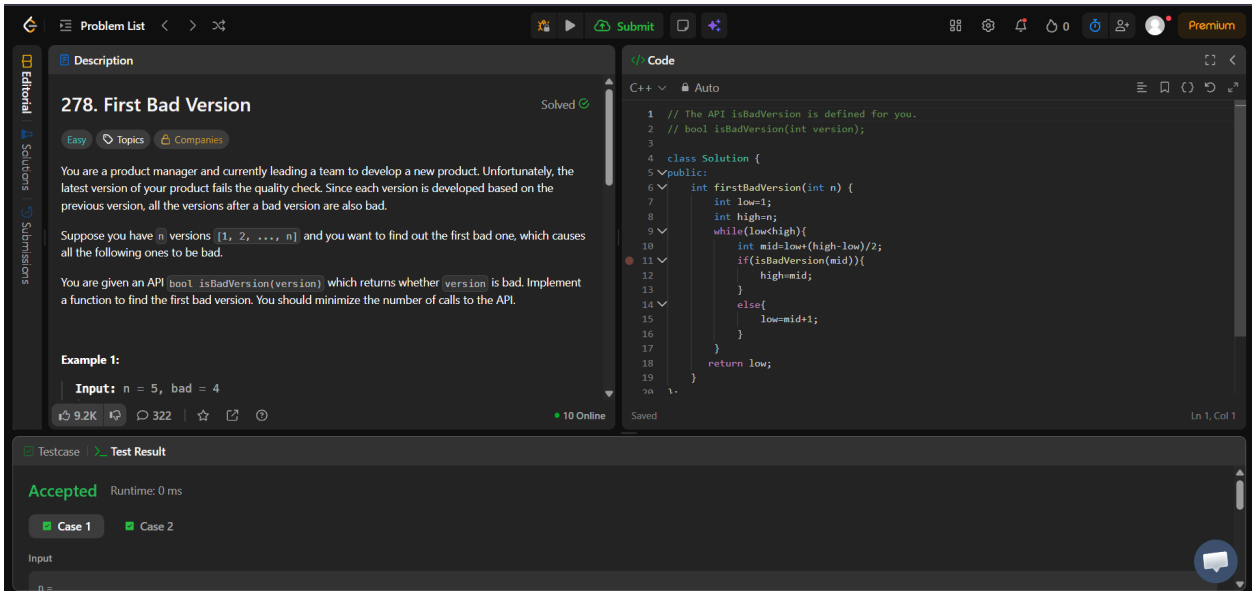
Explanation:
The target 2 should be inserted at index 1 to maintain sorted order.

Key Learning

If the target is not found, `low` identifies its correct insertion position after the loop finishes.

---

10. Optional LeetCode #278 — First Bad Version



Problem

There are `n` versions of a product. Once a version becomes bad, every version after it is also bad. Find the first bad version while minimizing calls to `isBadVersion()`.

Binary Search Logic

- If `mid` is bad, the first bad version may be `mid` or an earlier version. Set `high = mid`.
- If `mid` is good, the first bad version must be later. Set `low = mid + 1`.
- When `low == high`, that position is the first bad version.

C++ Solution

```
// The API isBadVersion(int version) is provided.

class Solution {
public:
```

```
    int firstBadVersion(int n) {
        int low = 1;
        int high = n;

        while (low < high) {
            int mid = low + (high - low) / 2;

            if (isBadVersion(mid)) {
                high = mid;
            } else {
                low = mid + 1;
            }
        }

        return low;
    }
};
```

Example

Input:

n = 5
bad = 4

Output:

4

Why `high = mid`?

The middle version may itself be the first bad version, so it must remain a candidate.

Complexity

- Time: O(log n)
- Auxiliary space: O(1)
- API calls: At most approximately ceil(log2(n)) for this implementation.

Status

Accepted — 24/24 test cases passed in the submitted solution.

---

11. Important Revision Questions

1. What is a function in C++?
2. What is the difference between a parameter and an argument?
3. What is the difference between pass by value and pass by reference?
4. What does the `&` symbol mean in a reference parameter?
5. What does the `*` symbol mean in a pointer declaration?
6. Why can a pointer parameter modify the original variable?
7. Why should the array size be passed to a function?
8. Why does Binary Search require sorted data?
9. Why is Binary Search O(log n)?
10. What is the difference between `low <= high` and `low < high` in these implementations?
11. Why does Search Insert Position return `low` if the target is absent?
12. Why does First Bad Version set `high = mid` when the middle version is bad?

---

12. Day 3 Summary

Concepts Learned

- Functions and reusable code.
- Parameters and arguments.
- Pass by value, address, and reference.
- Passing arrays to functions.
- Binary Search and its complexity.
- Search Insert Position.
- Finding the first bad version using Binary Search.

Problems Practised

- LeetCode #704 — Binary Search
- LeetCode #35 — Search Insert Position
- LeetCode #278 — First Bad Version (optional)

Main Takeaway

Functions improve code organization, parameter-passing methods control how functions access data, and Binary Search efficiently narrows a sorted search range in logarithmic time.